

软件设计基础与体系结构概念

软件工程与计算 II · 体系结构模块 · 第 1 讲

南京大学 · 刘钦

今日学习路径

① 为什么设计 ② 核心思想 ③ 理解设计 ④ 设计分层 ⑤ 体系结构定义 ⑥ AI增强

复杂性管理 分解与抽象 概念与演化 低/中/高层 三要素+风格初步 架构气味+ADR



今日核心问题

1. 软件设计的**本质**是什么？与编程、需求分析有什么关系？
2. 什么是软件体系结构？它由哪些**核心要素**组成？
3. AI 能辅助做出哪些**架构决策**？

预习测试

1. 软件设计与编程是什么关系？
2. 需求分析与软件设计是什么关系？
3. 什么是软件体系结构？

PART 1 · 为什么需要设计

事物的复杂性 vs 思维的有限性

讨论

- 什么是软件设计？
- 为什么要做设计？
 - 软件设计与编程是什么关系？
 - 需求分析与软件设计是什么关系？
- 怎么做设计？

软件的本质复杂性 [Brooks 1987]

"The complexity of software is an **essential** property, not an **accidental** one."

—— F. Brooks, **No Silver Bullet**

Why software is inherently complex?

- 问题域的复杂性 — 现实世界的规则、约束和例外
- 开发过程的管理难度 — 多人协作、需求变更
- 软件的灵活性 — 修改容易导致"什么都能改"的幻觉
- 离散系统的行为刻画 — 无法像连续系统那样用微分方程建模

如何应对复杂性？[SWEBOK 2.1]

人类认知的局限

- **7 ± 2 规则** (Miller, 1956): 人类短期记忆一次只能处理 5~9 个信息块
- 软件系统动辄成百上千个类——远超人脑容量

应对策略

策略	含义	设计中的体现
关注点分离	每次只关注一个维度	分层、模块化
分解	大问题拆成小问题	子系统、包
抽象	隐藏细节，暴露接口	接口、抽象类

设计的复杂度 [SWEBOK 2.1]

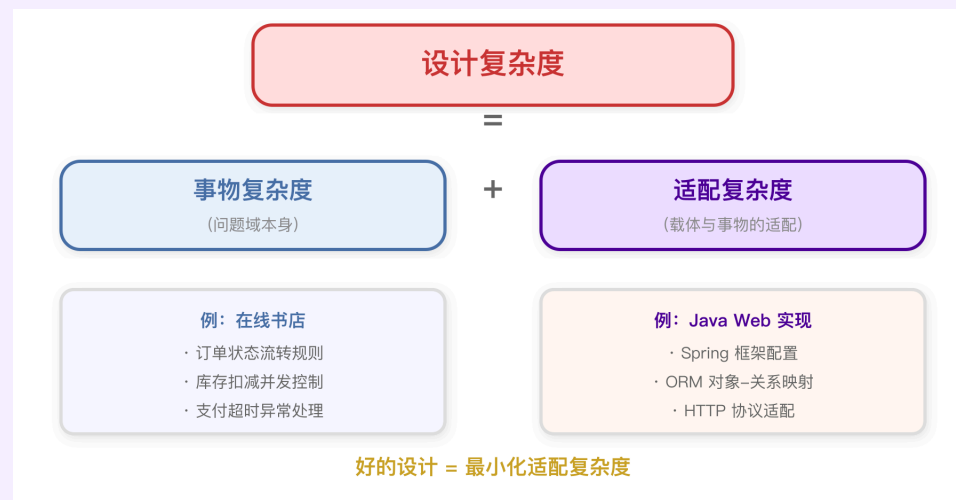
两种复杂度

- **事物复杂度** — 问题域本身的复杂性
- **载体复杂度** — 软件实现带来的额外复杂性

公式

设计复杂度 = 事物复杂度 + 载体与事物的适配复杂度

好的设计 = 最小化适配复杂度

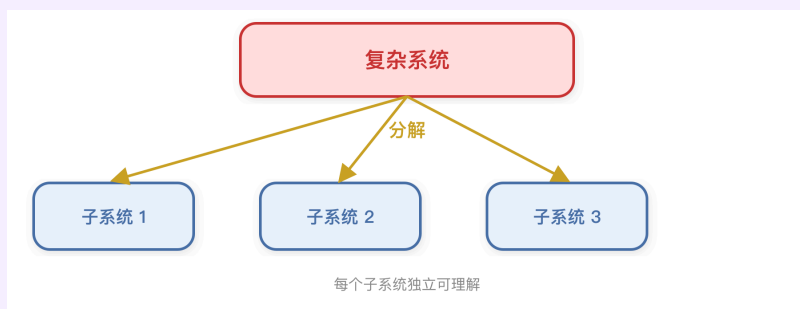


PART 2 · 软件设计的核心思想

分解与抽象

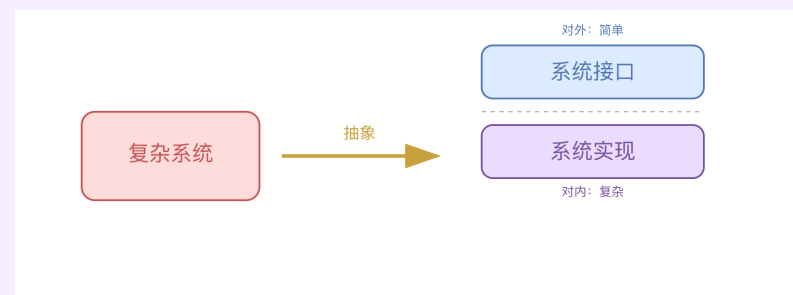
分解与抽象 —— 控制复杂性的两大武器 [SWEBOK 2.1]

分解 (Decomposition)



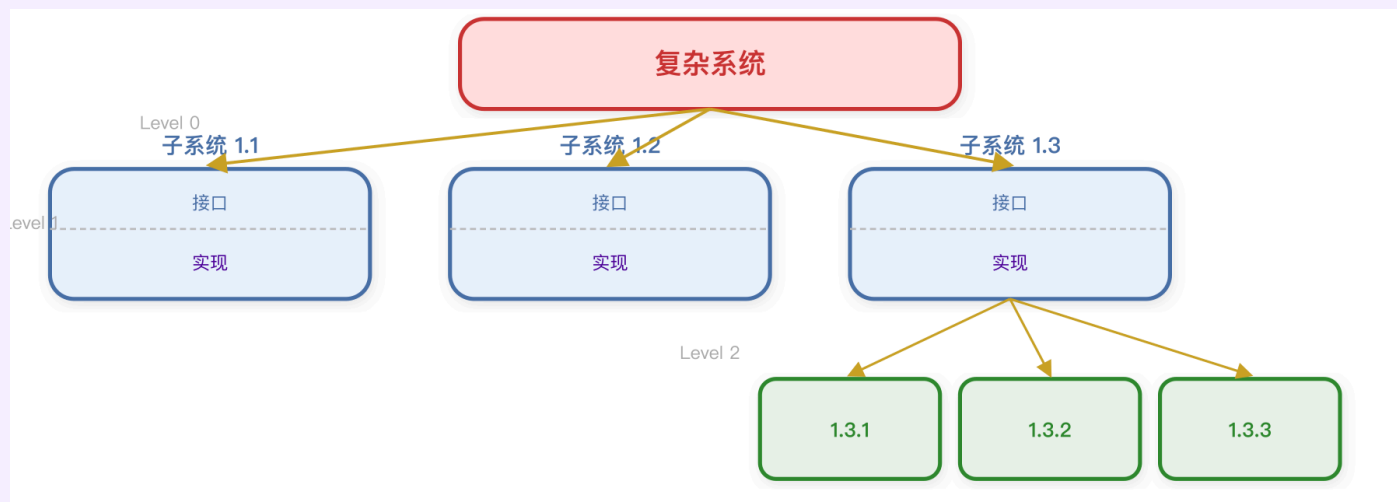
- 将大问题分而治之
- 每个子系统独立可理解

抽象 (Abstraction)



- 隐藏内部细节
- 暴露必要接口

分解与抽象的并用和层次性 [SWEBOK 2.1]



每一层都同时使用**分解**（拆成子系统）和**抽象**（每个子系统有接口和实现）

软件设计思想的发展

时代	关键词	核心思想
程序设计	程序、函数、过程	自顶向下、逐步求精
软件设计	抽象数据类型、信息隐藏、面向对象	独立于实现、封装变化
大规模软件设计	体系结构、设计模式、框架、构件、产品线	为复用而设计、重构

2010 年代以后

- 云原生时代：Docker、K8s、Istio — 微服务架构
- 大数据和 AI 时代：LLM 的引入可能是划时代的改变（AI for SE, SE for AI）

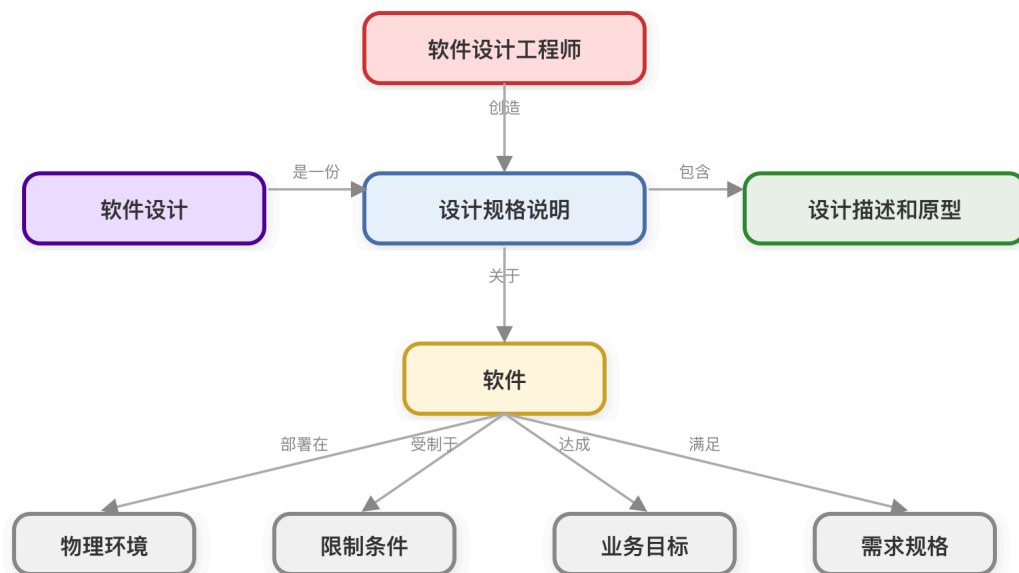
PART 3 · 理解软件设计

什么是设计？设计与编程、需求分析的关系

什么是设计？ [SWEBOK 2.1]

- [Wiki] Design is the planning that lays the basis for the making of every object or system.
- [Brooks 2010] 定义的精髓在于**计划、思维和后续执行**
- [Ralph 2009] 将设计定义为：
 - **设计（名词）**：一个对象的规格说明。由人创造，有明确目标，适用于特定环境
 - **设计（动词）**：在一个环境中创建对象的规格说明

软件设计 [SWEBOK 2.1]



软件设计的演化性 [SWEBOK 2.2]

设计不是一次完成的

- 设计决策会随着对问题域理解的加深而**不断修正**
- 需求变更会导致设计变更 — 这是**常态而非例外**

设计决策的概念完整性

概念完整性是系统设计中最重要考虑因素。

—— F. Brooks, **The Mythical Man–Month**

- 一个系统的所有设计决策应该反映**一致的理念**
- 避免"设计委员会"导致的**风格混杂**

设计、编程与需求分析的关系

维度	需求分析	软件设计	编程
回答的问题	做什么？	怎么组织？	怎么实现？
关注层面	问题域（现实世界）	解决方案的结构	代码细节
产出物	用例图、概念类图	包图、组件图、接口	Java/Python 代码
类的粒度	概念类（无方法）	设计类（有接口）	实现类（有全部代码）
可修改性	高	中（架构决策难改）	高（但受架构约束）

从需求到代码：概念类 → 设计类 → 实现类，逐步精化

PART 4 · 软件设计的分层

低层、中层、高层

软件设计的分层 [SWEBOK 2.3]

层次	关注点	产出物	案例
低层设计	单个类的属性、方法、算法	设计类图	<code>Order.calculateTotal()</code>
中层设计	包/模块的划分、类的协作	包图、交互图	<code>com.bookstore.service.order</code>
高层设计（体系结构）	系统整体结构、层次、部署	组件图、部署图	三层架构：展示层/业务层/数据层

本讲聚焦**高层设计**（体系结构），中层和低层设计将在后续讲次展开

设计的活动、方法与模型 [SWEBOK 2.4]

设计活动

1. **体系结构设计** — 确定系统的宏观结构
2. **接口设计** — 定义模块之间的契约
3. **组件设计** — 设计每个模块的内部结构
4. **数据设计** — 设计数据存储方案

设计方法

- **面向对象设计** (OOD)
- **结构化设计** (SD)
- **领域驱动设计** (DDD)

设计模型

- **静态模型**: 类图、包图、组件图
- **动态模型**: 顺序图、状态图、活动图

设计视角 —— 4+1 视图预告 [Kruchten 1995]

视图	关注点	UML 图
逻辑视图	功能划分、类和包的组织	类图、包图
开发视图	代码模块组织、构建管理	组件图
进程视图	并发、同步、性能	活动图
物理视图	部署、网络拓扑	部署图
场景视图	用例驱动，贯穿以上四个视图	用例图

4+1 视图模型将在 03-03 中结合案例详细展开

PART 5 · 软件体系结构的定义

部件 + 连接件 + 配置

软件体系结构的发展历程

年代	里程碑	关键人物/组织
1969	NATO 会议首次提出 "software architecture"	Ian P. Sharp
1972	"On the Criteria to Be Used in Decomposing Systems into Modules"	David Parnas
1992	"Foundations for the Study of Software Architecture"	Perry & Wolf
1995	4+1 视图模型	Philippe Kruchten
1996	Software Architecture: Perspectives on an Emerging Discipline	Shaw & Garlan
1998	Software Architecture in Practice (SEI 三部曲之一)	Bass, Clements, Kazman
2000	IEEE 1471 标准	IEEE/OMG

Perry & Wolf (1992): 经典公式

Software Architecture = { Elements, Forms, Rationale }

要素	含义
Elements	组成系统的处理元素、数据元素和连接元素
Forms	元素的组织形式和拓扑结构
Rationale	为什么选择这种元素和组织方式的理由

Barry Boehm 随后补充了 **Constraints** (约束条件)

Shaw (1995): 部件–连接件–配置模型

软件体系结构 = { 部件(Component), 连接件(Connector), 配置(Configuration) }

要素	定义	在 Java 中的映射
部件	承载系统主要功能的基本组成单位，包括处理与数据	包 (Package)、类集合
连接件	定义部件间交互的抽象表示	接口 (Interface)、事件
配置	定义部件和连接件之间的关联方式，组成系统总体结构	依赖注入、工厂模式

Bass, Clements & Kazman [SEI]

The software architecture of a system is the **set of structures** needed to reason about the system, which comprise **software elements**, **relations** among them, and **properties** of both.

Kruchten 的补充

Software Architecture encompasses the **significant decisions** about:

- the **organization** of a software system
- the selection of **structural elements** and their **interfaces**
- the **composition** of these elements into progressively larger subsystems
- the **architectural style** that guides this organization

案例 —— 在线书店系统的三要素映射 [SWEBOK 2.3]

部件 (Component)

```
com.bookstore.service.book  
com.bookstore.service.order  
com.bookstore.service.customer  
com.bookstore.dao
```

连接件 (Connector)

```
public interface BookService {  
    Book findById(String bookId);  
    List<Book> search(String kw);  
    boolean checkStock(  
        String bookId, int qty);  
}
```

配置 (Configuration)

```
// 依赖注入：配置部件间的拓扑  
BookService bookService =  
    new BookServiceImpl(bookDao);  
  
OrderService orderService =  
    new OrderServiceImpl(  
        orderDao, bookService);
```

映射总结

架构概念	Java 映射
部件	包 / 类集合
连接件	接口
配置	DI / 工厂

PART 6 · 体系结构的重要性

最难修改的设计决策

体系结构为什么重要？[Clements 1996]

三个根本理由

#	理由	说明
1	相互沟通	架构图是利益相关人之间的"共同语言"
2	早期设计决策	架构决定了系统的质量属性（性能、安全、可维护性）
3	可转移的抽象	好的架构可以在类似项目中复用

修改一个类的方法签名影响范围有限，
但把单体系统改为分层系统——几乎等于重写。

体系结构风格初步预告

风格	核心思想	典型应用
分层 (Layered)	每层只依赖下层	Web 三层架构
MVC	模型-视图-控制器分离	Spring MVC
主程序-子程序	自顶向下调用	简单脚本程序
面向对象	封装、继承、多态	Java 系统
微服务	独立部署的小服务	Netflix、Uber
事件驱动	发布-订阅解耦	消息队列系统

下一讲 (03-02) 将深入讲解分层风格和 MVC 风格，并结合案例演示

Conway 定律 [Conway 1968] [SWEBOK 1.3]

"Organizations which design systems are constrained to produce designs which are **copies of the communication structures** of these organizations."

案例：在线书店团队 → 模块划分

团队成员	负责模块	对应包
前端工程师 A	展示层	<code>controller</code>
后端工程师 B	图书 + 搜索	<code>service.book</code>
后端工程师 C	订单 + 支付	<code>service.order</code> , <code>service.payment</code>
后端工程师 D	用户 + 购物车	<code>service.customer</code> , <code>service.cart</code>
DBA E	数据层	<code>dao</code>

5 人团队 → 5 个模块边界。组织结构决定了架构边界。

利益相关人与关注点 [SWEBOK 1.2]

案例：在线书店 Stakeholder–Concern 矩阵

利益相关人	主要关注点	对架构的影响
顾客	可用性、响应速度、易用性	需要缓存层、CDN
开发团队	可维护性、可测试性	分层 + 接口隔离
运维	可部署性、可监控性	日志框架、健康检查端点
产品经理	功能完整性、上线速度	模块化便于并行开发
安全团队	数据安全、合规性	HTTPS、加密存储、权限控制
管理层	成本、进度	技术选型约束（Java + Spring Boot）

不同的利益相关人对**同一个系统**有不同的"什么是重要的"——这就是 **Concern**

PART 7 · AI 增强：架构决策与架构气味

LLM 辅助 ADR 生成 + Architecture Smell 识别

什么是架构气味（Architecture Smell）？

架构气味是系统体系结构中可能导致**维护困难**、**性能下降**或**演化受阻**的结构性问题。

气味类型	描述	后果
循环依赖	$A \rightarrow B \rightarrow C \rightarrow A$ 形成环	无法独立编译/部署/测试
God Component	一个包/模块承担过多职责	修改波及面大，测试困难
层违规调用	展示层直接访问数据层	绕过业务规则，安全风险
Feature Envy	模块频繁访问另一模块的内部数据	高耦合，应考虑职责迁移
Scattered Functionality	同一功能分散在多个模块	修改需同时改多处

AI Agent 时代的架构决策 [arxiv 2604.04990, 2025]

"Architecture Without Architects" — AI Coding Agent 正在**隐式地**做出架构决策

发现	影响
Agent 在 秒级 做出架构决策	决策速度远超人类，但缺乏深思熟虑
不同 LLM 生成 结构性不同 的代码	模型选择本身就是一个架构决策
决策 无记录 ，缺乏透明性	无法追溯架构 Rationale
Agent 的决策面覆盖 全架构维度	分层/模块划分/依赖关系/API 设计全被 Agent 决定

推荐实践

Humans own architecture, AI validates it.

人负责架构决策，AI 负责验证和生成。

架构气味自动检测工具链

工具	类型	能力
Arcan	静态分析	检测循环依赖、God Component、层违规调用
Sen4Smells	演化分析	基于版本历史计算架构技术债务指数
RABERT [2025]	Transformer	基于关系感知 BERT 增强代码/架构气味检测准确率
CodeAnt.ai	ML 平台	用机器学习排序气味优先级，聚焦高影响问题
DeepSource	CI/CD 集成	实时反馈 + 自动修复 PR 中的反模式

趋势：从规则驱动（正则匹配依赖违规）→ AI 驱动（Transformer 理解代码语义）

ADR —— 架构决策记录 [SWEBOK 2.4]

Architecture Rationale: 不仅记录决策, 还要记录为什么拒绝其他方案

字段	内容
标题	ADR-001: 采用三层分层架构
状态	已采纳
背景	在线书店需支持 Web 和移动端, 业务逻辑需独立于展示
决策	采用 Presentation → BusinessLogic → Data 三层架构
理由	分层降低耦合, 便于团队并行开发, 满足可维护性需求
被拒绝方案 1	微服务架构 — 拒绝理由: 团队仅 5 人, 无 K8s 运维经验
被拒绝方案 2	单文件脚本 — 拒绝理由: 无法扩展, 无法测试
后果	层间通过接口通信; 跨层调用需严格禁止

Architecture Rationale 捕获"为什么这样决策": 被考虑的替代方案、权衡和拒绝理由

LLM 辅助 ADR 生成

Prompt 示例

你是一名软件架构师。请根据以下需求和约束，生成一份 ADR（架构决策记录）。

系统：在线书店系统

功能需求：13 个用例（浏览、搜索、购买、管理等）

非功能需求：

- 响应时间 < 2 秒
- 支持 1000 并发用户
- 支付信息加密传输

项目约束：开发团队 5 人，使用 Java + Spring Boot

请输出标准 ADR 格式，包含：标题、背景、决策、理由、后果。

人工审查重点：理由是否充分？后果是否被低估？

案例 —— 让 LLM 识别架构气味

Prompt 示例

以下是在线书店系统的包依赖关系：

- presentation → businesslogic, data
- businesslogic → data
- data → businesslogic (反向依赖)

请识别其中的架构气味，并给出修复建议。

LLM 可能的输出

发现	类型	修复建议
data → businesslogic 反向依赖	循环依赖	提取接口到独立包，用依赖倒置
presentation → data 跨层调用	层违规	删除直接引用，通过 businesslogic 中转

AI 能快速发现依赖问题，但修复方案需人工判断权衡

架构技术债务（Architectural Technical Debt）[SWEBOK 2.4]

今天的架构决策可能在软件系统的整个生命周期中产生**显著的后续成本**。
被推迟的决策可能损害可维护性或未来的可演化性——这些债务终将需要偿还。

维度	示例	后果
模块化缺失	初期为赶进度不做分层	后续版本开发时间成倍增加
接口设计不足	层间直接引用实现类	替换数据库需要修改全部代码
文档缺失	不写 ADR	新成员无法理解架构决策理由
测试缺失	不做集成测试	架构退化无法被及时发现

架构技术债务可以用**视图和视点**来分析和管理的 [Kruchten 2019]

技术决策的长远影响

案例：高并发平台架构崩溃

- 某电商平台双十一期间**单体架构**承压崩溃，损失数亿
- 根因：早期架构决策未考虑**可伸缩性**，修复需要重写整个系统
- 教训：架构是**最难修改的设计决策**，初期投入不足后期代价巨大

思考

- 技术决策不仅是技术问题，更是**商业决策**和**社会责任**
- 架构师需要在**短期成本**和**长期可持续性**之间权衡
- AI 可以辅助分析，但**决策的责任始终在人**

PART 8 · 课堂总结

课堂总结

主题	核心要点
为什么设计	软件本质复杂，需要分解与抽象来管理
核心思想	分解（分而治之）+ 抽象（隐藏细节）
理解设计	设计 = 创建规格说明；设计具有演化性
设计分层	低层（类）→ 中层（包）→ 高层（体系结构）
体系结构定义	部件 + 连接件 + 配置；是最难修改的设计决策
AI 增强	LLM 辅助 ADR 生成 + 架构气味识别

课后作业

任务一：概念理解

1. 用自己的语言解释**软件体系结构的三要素**（部件、连接件、配置），并各举一个 Java 中的例子
2. 解释**设计与编程的区别**，为什么说"架构是最难修改的设计决策"？

任务二：AI 实操

3. 用 LLM 为你的课程项目生成一份 **ADR**（架构决策记录）
4. 将项目的包依赖关系输入 LLM，让它**识别架构气味**并给出修复建议

提交格式：ADR 文档 + 架构气味分析报告

下节课预告

03-02 · 体系结构风格与设计过程

内容	要点
分层风格深入	严格分层 vs 松散分层；三层架构案例
MVC 风格	Model-View-Controller + 观察者模式
设计过程	7 步过程：需求分析 → 选择风格 → 逻辑/物理设计 → 迭代
案例演示	在线书店系统 + 连锁超市系统的分层设计全过程

预习要求

- 阅读教材 第八章 8.2 节（分层体系结构风格）
- 思考：为什么分层是最常用的体系结构风格？

谢谢！

软件工程与计算 II · 03-01 · 软件设计基础与体系结构概念

南京大学 · 刘钦